JABBERPOINT REDESIGN ADVICE

Software Quality

Stefania Panainte, Beyza Olmez, Miriam Cerlikova

Table of Contents

1. Introduction 2
2. Identified Issues in the Current System 2
3. Design Pattern 1 – Observer 3
4. Design Pattern 2 – Command 3
5. Design Pattern 3 – Factory Method 5
6. Clean Code Improvements 5
7. Redesign Overview 6
8. Location of Patterns in the Class Diagram 6
9. Expected Improvements 7

1. Introduction

The purpose of this redesign is to improve the structure and maintainability of the JabberPoint system. The system analysis performed earlier identified several design issues, including tightly coupled components, large controller responsibilities, and unclear separation between responsibilities in the model and user interface.

To address these issues, three design patterns are applied in the redesigned architecture:

- Observer
- Command
- Factory Method

These patterns improve separation of concerns, reduce coupling between classes, and make the system easier to extend.

In addition to design patterns, several improvements based on principles from Clean Code are applied to improve readability and maintainability.

The redesign is represented in a class diagram which shows how the patterns are applied in the system.

2. Identified Issues in the Current System

The system analysis revealed several structural issues in the current implementation.

Model–View Coupling

The Presentation class interacts directly with the view component SlideViewerComponent. This creates tight coupling between the application logic and the user interface, making the system harder to modify and test.

Large Controller Responsibilities

The MenuController class currently performs multiple responsibilities, including creating menu items, handling user input, and executing presentation logic. This violates the Single Responsibility Principle and makes the class difficult to maintain.

Distributed Object Creation

The XMLAccessor class directly creates slide item objects such as TextItem and BitmapItem. This spreads object creation logic across the system and makes the code harder to extend.

Code Quality Problems

Additional issues were identified in the codebase:

• use of legacy collections such as Vector

• unclear naming and spelling mistakes in identifiers

• large classes performing multiple responsibilities

• rendering logic placed in the domain model

These issues reduce code readability and increase maintenance effort.

3. Design Pattern 1 – Observer

Goal

The Observer pattern is used to reduce coupling between the presentation model and the user interface.

Location in the System

The pattern is applied between the following components:

- Presentation
- SlideViewerComponent

Implementation

Presentation acts as the subject that maintains a list of observers. A new interface called PresentationObserver is introduced. Classes implementing this interface receive updates when the presentation state changes.

SlideViewerComponent implements the PresentationObserver interface and updates the displayed slide whenever the presentation changes.

When the current slide changes, the Presentation class notifies all observers.

This pattern is implemented in the current system and is visible in the class diagram.

Why Observer

Observer is used because changes in the presentation state must automatically update the user interface.

Why not Mediator

Mediator introduces an additional coordination class to manage communication between components. In this case the interaction is simple state notification, which matches the purpose of the Observer pattern more closely.

4. Design Pattern 2 – Command

Goal

The Command pattern is used to reduce the complexity of the MenuController and KeyController and to separate user actions from controller logic.

Location in the System

The pattern is applied to the interaction between:

- MenuController
- KeyController
- Presentation
- JabberPoint

Both MenuController and KeyController act as invokers and trigger user actions through the Command interface. This ensures that actions such as navigating slides, opening presentations, saving presentations, or exiting the application behave consistently regardless of whether the user interacts through the menu or the keyboard.

Implementation

A Command interface is introduced that defines an execute operation. Each user action is implemented as a separate command class.

Examples of command classes include:

• NextSlideCommand

• PreviousSlideCommand

• OpenPresentationCommand

• SavePresentationCommand

• ExitCommand

Each command contains a reference to either the Presentation or the JabberPoint object, depending on the responsibility of the command.

Commands that manipulate slides use the Presentation object, while system-level actions such as exiting the application use JabberPoint.

MenuController and KeyController no longer perform presentation logic directly. Instead, they execute command objects that encapsulate the requested operation.

This implementation is reflected in the current class diagram.

Why Command

Command encapsulates user actions as objects. This reduces the size of controller classes and improves maintainability by separating input handling from application logic.

Why not Chain of Responsibility

Chain of Responsibility is used when multiple objects may handle a request. In the JabberPoint system each user action maps to exactly one operation, making the Command pattern more suitable.

5. Design Pattern 3 – Factory Method

Goal

The Factory Method pattern is used to centralize the creation of slide item objects.

Location in the System

This pattern is applied in the slide item creation process within XMLAccessor.

Implementation

A factory class called SlideItemFactory is introduced. This factory creates slide item objects such as TextItem and BitmapItem based on the information read from the XML file.

XMLAccessor no longer directly creates slide items. Instead, it calls the factory method which returns an object of type SlideItem.

Why Factory Method

Factory Method encapsulates object creation and prevents parsing logic from depending directly on concrete slide item classes.

Why not Abstract Factory

Abstract Factory is used to create families of related objects. The JabberPoint system only requires creation of individual slide items, so Factory Method is more appropriate.

6. Clean Code Improvements

In addition to design patterns, several improvements based on Clean Code principles are applied.

Replace Legacy Collections

The Vector collection used in the Slide class is replaced with the modern List interface implementation.

Improve Naming

Several identifiers contained spelling mistakes or unclear names and have been corrected to improve readability.

Apply Single Responsibility Principle

Classes are structured to have one clear responsibility.

For example:

• MenuController manages menu structure and delegates actions to commands

• KeyController handles keyboard input and delegates actions to commands

Rendering Responsibility

Rendering responsibilities are primarily handled by SlideViewerComponent, which is responsible for displaying slides in the user interface.

The Slide class retains basic rendering-related methods such as draw() and getScale(), but it does not manage UI behavior. This ensures a clear separation between presentation data and user interface concerns without overcomplicating the design.

7. Redesign Overview

The redesigned architecture includes the following elements in the current system.

New interfaces:

• PresentationObserver

• Command

New classes:

• NextSlideCommand

• PreviousSlideCommand

• OpenPresentationCommand

• SavePresentationCommand

• ExitCommand

• SlideItemFactory

These elements are visible in the redesigned class diagram.

8. Location of Patterns in the Class Diagram

The redesigned class diagram shows the three patterns as follows.

Observer pattern

- Presentation maintains a list of observers implementing PresentationObserver.
- SlideViewerComponent implements the observer interface and updates the view when notified.

Command pattern

- MenuController and KeyController execute Command objects.
- Concrete command classes perform actions on either Presentation or JabberPoint.
- Presentation is responsible for slide-related actions, while JabberPoint is responsible for application-level control such as exiting the application.

Factory Method pattern

- SlideItemFactory creates objects of type SlideItem.
- XMLAccessor uses the factory instead of directly creating slide items.

- The class diagram reflects the implemented architecture, where the Observer pattern decouples the view, the Command pattern structures user input handling, and the Factory Method pattern centralizes object creation.

9. Expected Improvements

The proposed redesign improves several important software quality attributes.

Maintainability
The separation between model, view, and controller responsibilities becomes clearer, making the system easier to modify and maintain.

Extensibility
New commands or slide item types can be added without modifying existing classes, improving support for future functionality.

Readability
Smaller classes with clearer responsibilities make the architecture easier to understand for developers.